K.R. MANGALAM UNIVERSITY

THE COMPLETE WORLD OF EDUCATION

# Fundamentals Of Operating System Lab

# (ENCA252)

Lab File

Submitted by: Greaty Raghav
(2401201082)

Submitted to

Ms. Jyoti Yadav

**School of Engineering & Technology**

# OPERATING SYSTEM LAB – ASSIGNMENT 2

### Topic: Implementation of Banker's Algorithm for Deadlock Avoidance

---

## 1. Aim

To implement **Banker's Algorithm** and determine whether the system is in a **safe state**, and to find the **safe sequence of processes**.

---

## 2. Theory

Deadlock is a condition in which a set of processes are unable to proceed because each process is waiting for resources held by others.

**Banker's Algorithm** is a deadlock avoidance technique used by the operating system. It checks whether granting a resource request keeps the system in a safe state.

A system is said to be in a **safe state** if there exists a sequence of processes such that each process can complete execution without causing deadlock.

---

## 3. Algorithm

1. Input the number of processes and resource types.
2. Input Allocation Matrix, Maximum Matrix, and Available Resources.
3. Calculate the Need Matrix using:

   **Need = Maximum − Allocation**

4. Initialize:

- Work = Available ○ Finish = False for all processes

5. Find a process such that:

   **Need ≤ Work**

6. If found:
   - Add process to safe sequence ○ Update Work = Work + Allocation ○ Mark process as finished
7. Repeat until:
   - All processes are completed → System is SAFE ○ No process can proceed → System is UNSAFE

---

## 4. Code

```python
def calculate_need(max_matrix, allocation):
    n = len(max_matrix)
    m = len(max_matrix[0])

    need = [[0] * m for _ in range(n)]

    for i in range(n):
        for j in range(m):
            need[i][j] = max_matrix[i][j] - allocation[i][j]

    return need


def bankers_algorithm(n, m, allocation, max_matrix, available):
    need = calculate_need(max_matrix, allocation)

    print("\nNeed Matrix:")
    for row in need:
        print(row)

    finish = [False] * n
    safe_sequence = []
    work = available[:]

    while len(safe_sequence) < n:
        found = False

        for i in range(n):
            if not finish[i]:
                if all(need[i][j] <= work[j] for j in range(m)):
                    for j in range(m):
                        work[j] += allocation[i][j]

                    safe_sequence.append(f"P{i}")
                    finish[i] = True
                    found = True

        if not found:
            break

    if len(safe_sequence) == n:
        print("\nSystem is in SAFE state")
        print("Safe Sequence:", " -> ".join(safe_sequence))
    else:
        print("\nSystem is NOT in safe state")



# ========== MAIN ==========

n = int(input("Enter number of processes: "))
m = int(input("Enter number of resource types: "))

print("\nEnter Allocation Matrix:")
allocation = []
for i in range(n):
    row = list(map(int, input(f"P{i}: ").split()))
    allocation.append(row)

print("\nEnter Maximum Matrix:")
max_matrix = []
for i in range(n):
    row = list(map(int, input(f"P{i}: ").split()))
    max_matrix.append(row)

print("\nEnter Available Resources:")
available = list(map(int, input().split()))

bankers_algorithm(n, m, allocation, max_matrix, available)
```

---

## 5. Input Used

Number of Processes = 5

Number of Resource Types = 3

**Allocation Matrix**

| Process | A | B | C |
|---|---|---|---|
| P0 | 0 | 1 | 0 |
| P1 | 2 | 0 | 0 |
| P2 | 3 | 0 | 2 |
| P3 | 2 | 1 | 1 |
| P4 | 0 | 0 | 2 |

---

**Maximum Matrix**

| Process | A | B | C |
|---|---|---|---|
| P0 | 7 | 5 | 3 |
| P1 | 3 | 2 | 2 |
| P2 | 9 | 0 | 2 |
| P3 | 2 | 2 | 2 |
| P4 | 4 | 3 | 3 |

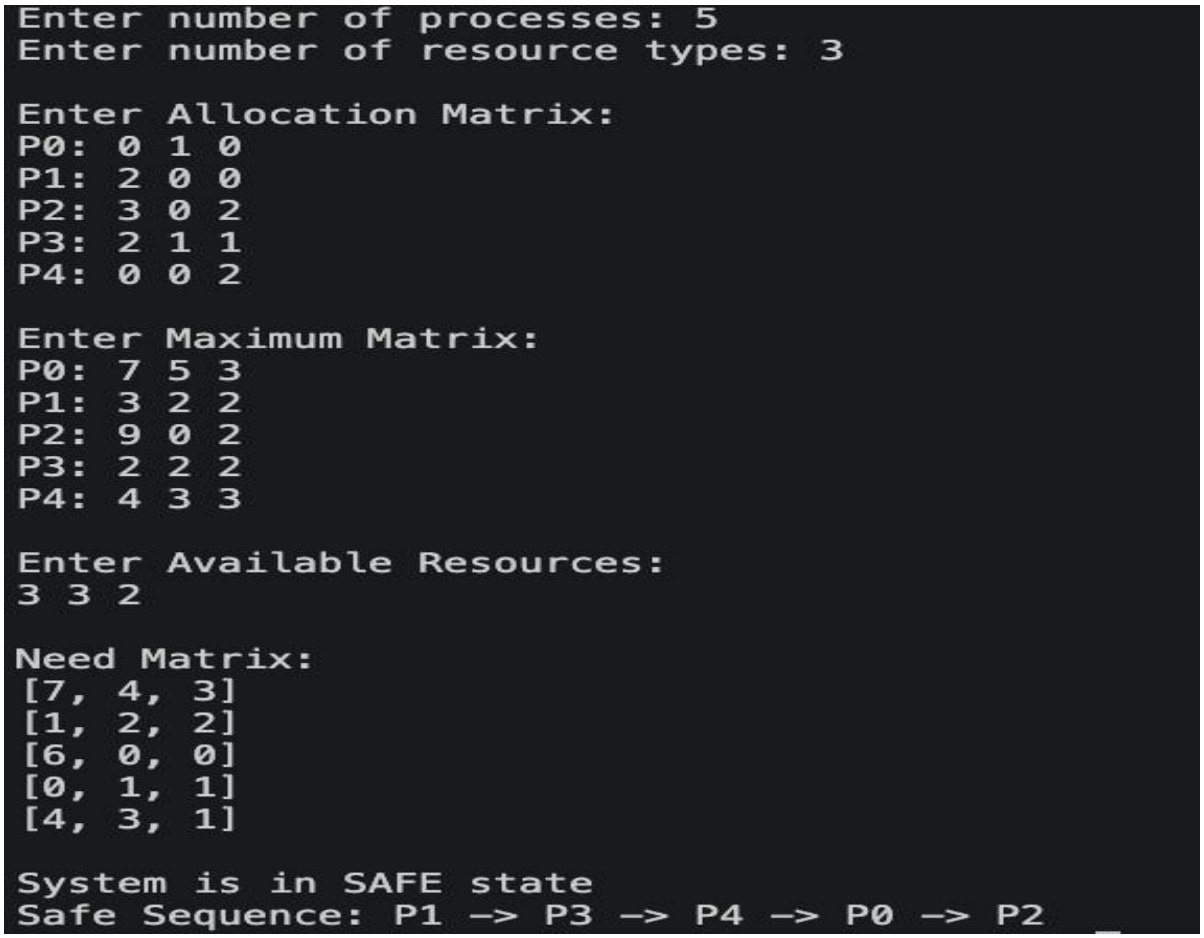
---

**Available Resources**

3 3 2

---

## 6. Output




---

## 7. Result Analysis

The Need Matrix is calculated using the formula:

**Need = Maximum − Allocation**

After applying Banker's Algorithm, the system is found to be in a **safe state** because a safe sequence exists.

The safe sequence obtained is:

**P1 → P3 → P4 → P0 → P2**

This indicates that all processes can complete execution without causing deadlock.

If no such sequence existed, the system would be considered unsafe.

Thus, Banker's Algorithm successfully prevents deadlock by ensuring safe resource allocation.

---

## 8. Conclusion

Banker's Algorithm is an effective method for deadlock avoidance.

It ensures that the system always remains in a safe state before allocating resources.

This algorithm helps in efficient resource management and prevents system failure due to deadlock.

---

## 9. GitHub Link

(https://github.com/Greaty01-Work/os-lab-assignments.git )